Final Route: The MD→ML Energy Predictor

- **RouteID:** Final
- **Wall:** Molecular Dynamics / Machine Learning
- **Grade:** ~ 5.10a
- **Estimated time:** 60 minutes
- **Routesetter:** William An K Do

Introduction

Why Do We Need Molecular Dynamics?

Proteins and all molecules do not exist in a vacuum. In reality, proteins rest inside cells in a variety of aqueous mediums such as the cytoplasm, blood, etc. Therefore, when we model proteins, we have to consider the various forces that a solvent and other molecules impose on other proteins of interest.

One of the techniques used to model solvent forces is Molecular Dynamics (MD). Molecular Dynamics simulations use classical mechanics to describe the forces exerted by the surrounding environment and the motion of atoms. By calculating these forces at very small time steps, MD simulations update the positions, velocities, and energies of the atoms throughout the system across each time step.

What Are We Doing Today?

In this route we'll be working with a program called OpenMM in Colab. In this routes, we'll introduce OpenMM so we can achieve the following learning objectives:

1. Understand the components that make up one OpenMM simulation
2. Run multiple OpenMM simulations
3. Analyze results/data from OpenMM
4. Use the data to train an ML model to predict the energy of the molecule

---

Exercise 0: The OpenMM Tools

Goal: Understand the Components of an OpenMM Stimulation

Step 1: Load OpenMM Package to Colab

Before anything, we need to download OpenMM to Colab:

```
!pip install openmm
```

Then we'll import 4 packages standard to OpenMM:

```
from openmm import app
import openmm as mm
from simtk import unit
import sys
```

This is everything a basic OpenMM simulation needs to run on Colab. Feel free to search OpenMM for additional packages for more advanced features.

Step 2: Loading our First Model and Other External Components

1. Go to my Github Link:

   https://github.com/Firearcher163/Final-Route-The-MD-ML-Energy-Predictor

and download the following files from it, both in the Ex 0-2 Materials folder

(There are also direct links)

   a. Xe-27.pdb
   b. Xenon.xml

2. Use `py3Dmol` to view the 3D molecule that will be ran in the OpenMM simulation

Step 3: Coding in the Five Tools in Every OpenMM Simulation

Here are the five tools every OpenMM Simulation has:

1. **The PDB Model (Topology)**
   - ➔ The atoms/molecules of interest, typically in the PDB format
   - ➔ For this first simulation, we'll use our 3x3x3 xenon cube (`xe-27.pdb`)

```
pdb = app.PDBFile("xe-27.pdb") # or any other pdb file
```

2. **The Force Field (.xml)**
   - ➔ A model of the forces between atoms that simulates the approximate interactions with solvents and other molecules.
   - ➔ For this first simulation, we'll use a xenon specific force field (Xenon.xml)

```
ff = app.ForceField('xenon.xml') # or any other xml file
```

3. **The System**
   - ➔ The physical model of the molecular system being simulated
   - ➔ Includes both the atoms and forces in the simulation (pdb + ff)

```
system = ff.createSystem(pdb.topology)
```

4. **The Integrator**
   - ➔ Updates the positions and velocities of atoms at each time step (set manually)
   - ➔ Three types: **Verlet** (basic), **Langevin** (temperature control), **Brownian** (Cramped)
   - ➔ For the Xenon cube, we'll use the **Verlet** with a **1 ferrosecond time step**

```
integrator = mm.VerletIntegrator(0.001*unit.picosecond)
```

5. **The Platform**
   - ➔ decides which hardware will run the simulation (similar to Colab's runtime)
   - ➔ Four types: **Reference** (Basic), **CPU** (CPU), **CUDA** (Nvidia GPU), **OpenCL** (AMD GPU)
   - ➔ For the Xenon cube, reference will suffice

```
platform = mm.Platform.getPlatformByName('Reference')
```

Now, with these five tools. We are ready to set up the OpenMM simulation.

E0 Success Check

- ☐ Displayed a 3D model of the xenon.pdb file
- ☐ Load and understand the five tools of an OpenMM

Exercise 1: Our First OpenMM Simulation

Goal: Set Up and Run an OpenMM Simulation of the 27 Xenon cube. Then analyze the data an OpenMM simulation provides.

Step 1: Set Up the OpenMM Simulation

With these five OpenMM tools, we can input all of them into `app.Simulation` to start an OpenMM simulation:

```
simulation = app.Simulation(pdb.topology, system, integrator, platform)
```

There's a 6th and final tool, **context,** but it requires the `app.Simulation` to be loaded in. Context sets the initial positions of the atoms at the start of the OpenMM simulation based on the pdb file. The **system** and **integrator** act upon this initial **context.** We can also use `minimizeEnergy` to set an initial baseline model at the lowest energy.

```
simulation.context.setPositions(pdb.positions)
simulation.minimizeEnergy()
```

Now, we gathered all the tools and we can begin our first simulation. We run simple code and the MD simulation will run. However, we need a way to gather data from this situation.

```
simulation.steps(100000) # or any number of steps you like
```

Step 2: Gathering OpenMM Data

There are three ways to acquire data.

1. `simulation.context.getState`
2. `app.StateDataReporter`
3. `app.PDBReporter`

`simulation.context.getState` takes a snapshot of the current state and collects the data at that state. This will be our way of acquiring the positions, velocities, and forces at a given state. To get these three at all states, create a for loop for all states.

```
state = simulation.context.getState(getPositions=True,
getVelocities=True, getForces=True)
```

```
positions = state.getPositions()
velocities = state.getVelocities()
forces = state.getForces()
```

`app.StateDataReporter` also collects data at a state, but it will collect the data at each time step for all states during the simulation. For this route, we'll take the step, temperature, totalEnergy and density using this command. "data.csv" is the output file. If you want a live version of the simulation you can use `sys.stdout` instead of data.csv. The number represents the step intervals before the simulation will record the data.

```
simulation.reporters.append(app.StateDataReporter("data.csv",1000,
step=True, temperature=True, density=True,totalEnergy=True))
```

`app.PDBReporter` does the same thing as `simulation.context.getState` but the data is saved as a pdb file. We won't use this method too much in this route.

Step 3: Your Turn

Run an OpenMM simulation that computes the following for Xe-27.pdb.

1. Computes measurements every **1000 steps for a total of 1,000,000 steps** (1000 measurements).

2. Computes the positions, velocities, and forces at each interval and stores them into lists.

3. Computes the temperature, density, and total energy at each interval.

E1 Success Check

- [ ] A .csv file was created containing the temperature, density, and total energies at each interval
- [ ] Three lists were created containing positions, velocities, and forces
- [ ] Each list contains 1000 entries (1 per interval)

Exercise 2: Analyzing OpenMM Data

Goal: Now that we have data, let's analyze it to see what we can do with OpenMM results.

Step 1: Set Up a Large Dataframe/Table

Task: Set up a Large Dataframe containing all of the `simulation.context.getState` and the `app.StateDataReporter` variables.

E2 Questions:

1. You may notice that positions, velocities, and forces are presented as a list in a list in a list (a 3-level nested list). What exactly does the data in these three columns represent and how is it organized?
2. Why would it be more challenging running OpenMM simulations on large proteins?

Hint: The last index in this 3-level nested list is `all_positions[999][26][2]`

Step 2: Plotting Fluctuations in the Data

Task: Plot both a histogram and a scatter plot for temperature, density, and total energy as a function of steps. Also, calculate the mean measurement for each variable.

E2 Questions

1. How do each of the variables fluctuate throughout the simulation?
2. Which plot do you prefer to showcase the variable fluctuations in a simulation

Step 3: Using the Data to Solve Chemical Problems

OpenMM allows us to generate data that can be used to solve other problems in chemistry.

Task: Assuming Xe-27 is an ideal gas, what is the pressure of the Xe-27 cube as a gas? *Hint:* Use Ideal Gas Law: PV = nRT

Exercise 3: Peptide OpenMM

Goal: Run OpenMM Simulations on real PDB files from the RCSB PDB.

Now that we understand the basics of OpenMM, let's apply OpenMM to a biochemical problem. Our overall goal is to train a ML model to predict the energy of small peptides from the pdb. However, those energies and features must come from somewhere, and they'll come from OpenMM. I compiled a bunch of small PDB files from RCSB PDB (six residues or less). Your goal is to run these PDB files on OpenMM to generate testing and training data for our ML model to predict energy.

This is our overall question. Is OpenMM data enough to make an accurate prediction of the energy of a molecule?

Step 1: Decide Which PDBs Are Compatible with the Amber Field

Unfortunately, the amber force amber14-all.xm used in proteins is very selective as to which PDB are compatible with it. Therefore, our code must also select which of the pdb files can run the simulation. All files for this exercise are in the Github folder Ex 3-5 Materials or in the overall Github link shown in exercise 1.

1. Load PDB_Six_Peptides.zip (You can unzip manually or in Colab)
2. Choose the parameters (OpenMM tools) for the upcoming 10 OpenMM simulations.
    a. Topology: PDB_Six_Peptides.zip
    b. Force Field: Load these two files; amber14-all.xml & tip3p.xml
        i. Both xml files should be loaded in. These Amber forces are specific for proteins and peptides. Tip3p represents water.
    c. System (Done Later)
    d. Integrator (Recommend: Langevin at room temperature 298.15K)
        i. Each simulation must have its own integrator
    e. Platform (Recommend: CPU or higher)
3. Use the PDB/ENT (both work) files to create pdb, systems, and simulations.
        i. Most of the pdb files don't work since the pdb format doesn't match the amber force field specifications. (*19 of the 44 entries will work*)
        ii. Make sure the Code doesn't break/produces an error

Step 2: Run OpenMM on the Working 19 PDB Files

1. Create 19 simulations (one for each entry) and set up the reporter/data collectors to measure the positions, velocities, forces, and total energy every 1000 steps for 50,000 total steps (500 measurements per simulation)

2. Run the 19 OpenMM simulations. In the end, there should be a total of 9500 measurements taken (500 per entry).

3. Save the positions, velocities, forces, and total energy in a convenient file to load for exercise 4.

E3 Success Check

- [ ] Data file saved containing the OpenMM positions, velocities, forces, and total energy
- [ ] Simulations for all 19 compatible entries were completed
- [ ] 9500 total measurements were taken

Exercise 4: The OpenMM ML Model for Energy Prediction

Goal: Use the OpenMM data and other structural features to create a randomforest ML model that predicts the energy of structures.

Although, this task is straightforward. Here are some things to note while constructing the ML randomforest model.

1. The features for the matrix will include the OpenMM positions, velocities, and forces for every atom. Also include the number of atoms, number of bonds, and molecular weight of the entry.

2. Since each entry has a different number of atoms, we need to do '**padding**' to make sure every input feature vector has the same length. Use the chatbot to understand what padding is and how to do padding.

3. Cluster the data based on the entries. Let 15 entries encompass the training data while 4 entries encompass the testing data. To do this look up `from sklearn.preprocessing import StandardScaler` and `from sklearn.cluster import KMeans` to cluster data based on which entry is from. This is to prevent leakage as frames data in one entry tend to be similar.

4. Once the model is trained, evaluate the model by calculating R^2 and RMSE.

E4 Question

1. Was OpenMM data useful in predicting the energy of a molecule? Why or why not?

Exercise 5 "Optimal Stretch Goal": Frames vs Entries

Goal: Run an OpenMM with Less Entries and More Frames and Evaluate which Method is Better.

In exercise3, we ran 19 entries and each entry had 50 frames of measured data. Let's flip that.

Task: Run an OpenMM simulation on 5 entries, but each entry has 200 frames of measured data (20000 total steps and 1000 step intervals). Then, use the same procedure in exercise 4 to train a randomforest model and compare the models in exercise 4 and 5.

E5 Question

1. Which model produces better results, more entries or more frames? Why do you think that was the case?

Success You Have Completed this Route!

Note: Answers to all the questions are in the notebook.